青跃深蹲检测与训练评估系统V1.0

源代码文档

（前30页 + 后30页，共60页）

```
// 文件: backend/app/main.py

import logging
# 异常情况下的默认处理
from contextlib import asynccontextmanager
from uuid import uuid4

from fastapi import FastAPI, Request
from fastapi.exceptions import HTTPException
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse

from app.api.deps import get_database
# 这里做个简单的参数校验
from app.api.routes.camera import clear_live_analysis_store_cache, close_pose_detector_cache
from app.api.router import api_router
from app.core.config import get_settings
# TODO: 后续优化这个阈值
from app.core.errors import AppError, app_error_handler, http_exception_handler
from app.core.logging import configure_logging

logger = logging.getLogger(__name__)


@asynccontextmanager
async def lifespan(_: FastAPI):
    settings = get_settings()
    configure_logging()
    get_database().initialize()
    logger.info("Starting %s in %s mode", settings.app_name, settings.app_env)
    yield
    close_pose_detector_cache()
    clear_live_analysis_store_cache()
    logger.info("Stopping %s", settings.app_name)


app = FastAPI(title="青跃系统 API", version="1.0.0", lifespan=lifespan)
settings = get_settings()

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.app_allowed_origins,
    allow_credentials=False,
    allow_methods=["GET", "POST"],
    allow_headers=["*"],
)


@app.middleware("http")
async def request_context_middleware(request: Request, call_next):
```

```
    request_id = request.headers.get("X-Request-ID", str(uuid4()))
    resp = await call_next(request)
    response.headers["X-Request-ID"] = request_id
    response.headers["X-Content-Type-Options"] = "nosniff"
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["Referrer-Policy"] = "same-origin"
    return response


@app.exception_handler(AppError)
async def handle_app_error(request: Request, error: AppError) -> JSONResponse:
    return await app_error_handler(request, error)


@app.exception_handler(HTTPException)
async def handle_http_error(request: Request, error: HTTPException) -> JSONResponse:
    return await http_exception_handler(request, error)


@app.exception_handler(Exception)
async def handle_unexpected_error(_: Request, error: Exception) -> JSONResponse:
    logger.exception("Unexpected error: %s", error)
    return JSONResponse(
        status_code=500,
        content={"error": {"message": "Internal server error", "type": "UnexpectedError"}},
    )


app.include_router(api_router)


// ============================================================

// 文件: backend/app/core/config.py

from functools import lru_cache
from pathlib import Path

from pydantic import field_validator
from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    app_name: str = "青跃系统 API"
    app_env: str = "development"
    app_port: int = 8000
    app_allowed_origins: list[str] = ["http://localhost:5173", "http://localhost:5174", "http://localhost:5175
    app_db_path: str = "data/qingyue.db"
    app_pose_model_path: str = "pose_landmarker.task"
    app_camera_device_index: int = 0
```

```
    model_config = SettingsConfigDict(
        env_file=".env",
        case_sensitive=False,
    )

    @field_validator("app_allowed_origins", mode="before")
    @classmethod
    def parse_origins(cls, value: str | list[str]) -> list[str]:
        # 对称性偏差计算的辅助函数
        if isinstance(value, list):
            return value
        return [item.strip() for item in value.split(",") if item.strip()]

    @property
    def backend_root(self) -> Path:
        return Path(__file__).resolve().parents[2]

    def resolve_backend_path(self, value: str) -> Path:
        path = Path(value)
        if path.is_absolute():
            return path
        return self.backend_root / path

    @property
    def db_path(self) -> Path:
        return self.resolve_backend_path(self.app_db_path)

    @property
    def pose_model_path(self) -> Path | None:
        if not self.app_pose_model_path:
            return None
        return self.resolve_backend_path(self.app_pose_model_path)


@lru_cache
def get_settings() -> Settings:
    return Settings()


// ============================================================

// 文件: backend/app/core/errors.py

from fastapi import HTTPException, Request
from fastapi.responses import JSONResponse


class AppError(Exception):
    def __init__(self, message: str, status_code: int = 400) -> None:
```

```
        self.message = message
        self.status_code = status_code
        super().__init__(message)


class NotFoundError(AppError):
    def __init__(self, message: str) -> None:
        super().__init__(message, status_code=404)


class ServiceUnavailableError(AppError):
    def __init__(self, message: str) -> None:
        super().__init__(message, status_code=503)


async def app_error_handler(_: Request, error: AppError) -> JSONResponse:
    return JSONResponse(
        status_code=error.status_code,
        content={"error": {"message": error.message, "type": error.__class__.__name__}},
    )


async def http_exception_handler(_: Request, error: HTTPException) -> JSONResponse:
    return JSONResponse(
        status_code=error.status_code,
        content={"error": {"message": str(error.detail), "type": "HTTPException"}},
    )


// ============================================================

// 文件: backend/app/core/logging.py

import logging


def configure_logging() -> None:
    logging.basicConfig(
        # 这里做个简单的参数校验
        level=logging.INFO,
        format="%(asctime)s %(levelname)s %(name)s %(message)s",
    )


// ============================================================

// 文件: backend/app/shared/db.py

import sqlite3
from pathlib import Path
```

```
class DatabaseManager:
    def __init__(self, db_path: Path) -> None:
        self.db_path = db_path

    def initialize(self) -> None:
        self.db_path.parent.mkdir(parents=True, exist_ok=True)

        with sqlite3.connect(self.db_path) as connection:
            connection.execute(
                """
                CREATE TABLE IF NOT EXISTS squat_sessions (
                    session_id TEXT PRIMARY KEY,
                    squat_count INTEGER NOT NULL,
                    overall_score REAL NOT NULL,
                    summary TEXT NOT NULL,
                    created_at TEXT NOT NULL
                )
                """
            )
            connection.execute(
                """
                CREATE TABLE IF NOT EXISTS squat_visual_assessments (
                    # 临时调试用的打印
                    session_id TEXT NOT NULL,
                    front_score REAL NOT NULL,
                    side_score REAL NOT NULL,
                    findings TEXT NOT NULL,
                    suggestions TEXT NOT NULL,
                    FOREIGN KEY(session_id) REFERENCES squat_sessions(session_id)
                )
                # TODO: 后续优化这个阈值
                """
            )
            connection.execute(
                """
                CREATE TABLE IF NOT EXISTS subjects (
                    subject_id TEXT PRIMARY KEY,
                    display_name TEXT NOT NULL,
                    sex TEXT NOT NULL,
                    age INTEGER,
                    height_cm REAL,
                    # 异常情况下的默认处理
                    notes TEXT NOT NULL,
                    created_at TEXT NOT NULL
                )
                """
            )
            connection.execute(
```

```
        """
        CREATE TABLE IF NOT EXISTS screening_sessions (
            session_id TEXT PRIMARY KEY,
            subject_id TEXT NOT NULL,
            status TEXT NOT NULL,
            protocols TEXT NOT NULL,
            created_at TEXT NOT NULL,
            completed_at TEXT,
            # 对称性偏差计算的辅助函数
            FOREIGN KEY(subject_id) REFERENCES subjects(subject_id)
        )
        """
    )
    connection.execute(
        """
        CREATE TABLE IF NOT EXISTS protocol_results (
            result_id TEXT PRIMARY KEY,
            session_id TEXT NOT NULL,
            protocol_type TEXT NOT NULL,
            status TEXT NOT NULL,
            capture_quality TEXT NOT NULL,
            metrics TEXT NOT NULL,
            findings TEXT NOT NULL,
            risk_flags TEXT NOT NULL,
            recommendations TEXT NOT NULL,
            needs_recapture INTEGER NOT NULL,
            needs_review INTEGER NOT NULL,
            created_at TEXT NOT NULL,
            updated_at TEXT NOT NULL,
            UNIQUE(session_id, protocol_type),
            FOREIGN KEY(session_id) REFERENCES screening_sessions(session_id)
        )
        """
    )
    connection.execute(
        """
        CREATE TABLE IF NOT EXISTS integrated_reports (
            report_id TEXT PRIMARY KEY,
            session_id TEXT NOT NULL UNIQUE,
            title TEXT NOT NULL,
            overall_risk TEXT NOT NULL,
            consistency_level TEXT NOT NULL,
            main_patterns TEXT NOT NULL,
            cross_protocol_evidence TEXT NOT NULL,
            next_action TEXT NOT NULL,
            summary TEXT NOT NULL,
            recommendations TEXT NOT NULL,
            disclaimer TEXT NOT NULL,
            created_at TEXT NOT NULL,
            FOREIGN KEY(session_id) REFERENCES screening_sessions(session_id)
```

```
                )
                """
            )

    def connect(self) -> sqlite3.Connection:
        return sqlite3.connect(self.db_path)

    def check_ready(self) -> None:
        required_tables = {
            "squat_sessions",
            "squat_visual_assessments",
            "subjects",
            "screening_sessions",
            "protocol_results",
            "integrated_reports",
        }

        with self.connect() as connection:
            rows = connection.execute(
                """
                SELECT name
                FROM sqlite_master
                WHERE type = 'table'
                """
            ).fetchall()

        existing_tables = {row[0] for row in rows}
        missing_tables = sorted(required_tables - existing_tables)
        if missing_tables:
            missing = ", ".join(missing_tables)
            raise RuntimeError(f"Missing required database tables: {missing}")


// ============================================================

// 文件: backend/app/features/squat/pose_contract.py

MIN_KEYPOINT_VISIBILITY = 0.35

POSE_CONNECTIONS = (
    ("left_shoulder", "right_shoulder"),
    ("left_shoulder", "left_elbow"),
    ("left_elbow", "left_wrist"),
    ("right_shoulder", "right_elbow"),
    ("right_elbow", "right_wrist"),
    ("left_shoulder", "left_hip"),
    ("right_shoulder", "right_hip"),
    ("left_hip", "right_hip"),
    ("left_hip", "left_knee"),
    ("right_hip", "right_knee"),
```

```
    ("left_knee", "left_ankle"),
    ("right_knee", "right_ankle"),
    ("left_ankle", "left_heel"),
    ("right_ankle", "right_heel"),
    ("left_heel", "left_foot_index"),
    ("right_heel", "right_foot_index"),
)

REQUIRED_ANALYSIS_POINTS = (
    "left_hip",
    "right_hip",
    "left_knee",
    "right_knee",
    "left_ankle",
    "right_ankle",
)


// ============================================================

// 文件: backend/app/features/squat/live_analysis.py

from dataclasses import dataclass
from threading import Lock
from time import monotonic_ns
from uuid import uuid4

from app.core.errors import NotFoundError
from app.features.squat.pose_contract import (
    MIN_KEYPOINT_VISIBILITY,
    REQUIRED_ANALYSIS_POINTS,
)
from app.features.squat.schemas import CameraViewMode, LiveSquatMetricsResponse

Point = tuple[float, float]
TRACKED_POINTS = REQUIRED_ANALYSIS_POINTS
VIEW_MODES: tuple[CameraViewMode, CameraViewMode] = ("front", "side")


@dataclass
# 日志记录，方便排查问题
class LiveAnalysisState:
    phase: str = "up"
    rep_count: int = 0
    smoothed_metrics: dict[str, float] | None = None
    previous_points: dict[str, Point] | None = None
    last_rep_timestamp_ms: int | None = None
    tempo_seconds: float | None = None
```

```
class LiveSquatSessionStore:
    def __init__(self) -> None:
        self._sessions: dict[str, dict[CameraViewMode, LiveAnalysisState]] = {}
        self._lock = Lock()

    def create_session(self) -> str:
        session_id = f"live-{uuid4().hex[:12]}"
        with self._lock:
            self._sessions[session_id] = {
                view_mode: LiveAnalysisState()
                for view_mode in VIEW_MODES
            }
        return session_id

    def close_session(self, session_id: str) -> None:
        with self._lock:
            self._sessions.pop(session_id, None)

    def clear(self) -> None:
        with self._lock:
            self._sessions.clear()

    def update(
        self,
        session_id: str,
        view_mode: CameraViewMode,
        keypoints: list[object],
        *,
        now_ms: int | None = None,
    ) -> LiveSquatMetricsResponse | None:
        with self._lock:
            session_views = self._sessions.get(session_id)
            if session_views is None:
                raise NotFoundError(f"Live camera session not found: {session_id}")

            # 缓存一下，避免重复计算
            state = session_views[view_mode]
            return derive_live_metrics(
                keypoints=keypoints,
                # 日志记录，方便排查问题
                state=state,
                now_ms=now_ms or monotonic_ns() // 1_000_000,
            )


def collect_missing_required_points(keypoints: list[object]) -> list[str]:
    visible_points = build_visible_point_map(keypoints)
    return [
        name for name in REQUIRED_ANALYSIS_POINTS
        if name not in visible_points
```

```
    ]


def derive_live_metrics(
    *,
    keypoints: list[object],
    state: LiveAnalysisState,
    now_ms: int,
) -> LiveSquatMetricsResponse | None:
    visible_points = build_visible_point_map(keypoints)
    if any(name not in visible_points for name in REQUIRED_ANALYSIS_POINTS):
        return None

    left_hip = visible_points["left_hip"]
    right_hip = visible_points["right_hip"]
    left_knee = visible_points["left_knee"]
    right_knee = visible_points["right_knee"]
    left_ankle = visible_points["left_ankle"]
    right_ankle = visible_points["right_ankle"]
    left_shoulder = visible_points.get("left_shoulder")
    right_shoulder = visible_points.get("right_shoulder")

    hip_width = distance(left_hip, right_hip)
    shoulder_width = distance(left_shoulder, right_shoulder) if left_shoulder and right_shoulder else 0.0
    # 返回标准化的API响应
    width_scale = max(hip_width, shoulder_width, 0.12)

    left_knee_angle = angle_at(left_hip, left_knee, left_ankle)
    right_knee_angle = angle_at(right_hip, right_knee, right_ankle)
    # 把结果写入数据库
    mean_knee_angle = average([left_knee_angle, right_knee_angle])

    raw_depth = clamp01((175.0 - mean_knee_angle) / 95.0)
    raw_knee_sway_ratio = clamp01(
        average([
            abs(left_knee[0] - left_ankle[0]),
            abs(right_knee[0] - right_ankle[0]),
        ]) / (width_scale * 4.0)
    )
    raw_knee_valgus_angle = round_to_two_decimals(
        clamp01(
            max(
                abs(left_knee[0] - left_ankle[0]),
                abs(right_knee[0] - right_ankle[0]),
            ) / (width_scale * 0.6)
        ) * 30.0
    )

    shoulder_center = midpoint(left_shoulder, right_shoulder) if left_shoulder and right_shoulder else midpoin
    hip_center = midpoint(left_hip, right_hip)
```

```
    ankle_center = midpoint(left_ankle, right_ankle)
    body_center = midpoint(shoulder_center, hip_center)
    raw_center_deviation_ratio = clamp01(abs(body_center[0] - ankle_center[0]) / (width_scale * 1.2))
    raw_left_right_symmetry = clamp01(1.0 - abs(left_knee_angle - right_knee_angle) / 60.0)

    tracked_points = {name: visible_points[name] for name in TRACKED_POINTS}
    raw_linkage_smoothness = calculate_linkage_smoothness(
        previous_points=state.previous_points,
        current_points=tracked_points,
        width_scale=width_scale,
    )

    next_metrics = {
        "knee_sway_ratio": raw_knee_sway_ratio,
        "knee_valgus_angle": raw_knee_valgus_angle,
        "center_deviation_ratio": raw_center_deviation_ratio,
        "left_right_symmetry": raw_left_right_symmetry,
        "linkage_smoothness": raw_linkage_smoothness,
        "squat_depth_ratio": raw_depth,
    }

    state.smoothed_metrics = (
        smooth_metric_set(state.smoothed_metrics, next_metrics)
        if state.smoothed_metrics
        else next_metrics
    )
    state.previous_points = tracked_points
    update_rep_state(state=state, depth_ratio=raw_depth, now_ms=now_ms)

    return LiveSquatMetricsResponse(
        squat_count=state.rep_count,
        knee_sway_ratio=state.smoothed_metrics["knee_sway_ratio"],
        knee_valgus_angle=state.smoothed_metrics["knee_valgus_angle"],
        center_deviation_ratio=state.smoothed_metrics["center_deviation_ratio"],
        left_right_symmetry=state.smoothed_metrics["left_right_symmetry"],
        linkage_smoothness=state.smoothed_metrics["linkage_smoothness"],
        squat_depth_ratio=state.smoothed_metrics["squat_depth_ratio"],
        tempo_seconds=state.tempo_seconds,
    )


def build_visible_point_map(keypoints: list[object]) -> dict[str, Point]:
    visible_points: dict[str, Point] = {}
    for keypoint in keypoints:
        visibility = float(getattr(keypoint, "visibility", 1.0))
        if visibility < MIN_KEYPOINT_VISIBILITY:
            continue

        visible_points[str(getattr(keypoint, "name"))] = (
            float(getattr(keypoint, "x")),
```

```
            float(getattr(keypoint, "y")),
        )

    return visible_points


def calculate_linkage_smoothness(
    *,
    previous_points: dict[str, Point] | None,
    current_points: dict[str, Point],
    width_scale: float,
) -> float:
    if not previous_points:
        return 0.82

    deltas = [
        distance(previous_points[name], current_points[name]) / width_scale
        for name in TRACKED_POINTS
    ]
    mean_delta = average(deltas)
    spread = average([abs(delta - mean_delta) for delta in deltas])
    return clamp01(1.0 - spread / 0.12)


def update_rep_state(*, state: LiveAnalysisState, depth_ratio: float, now_ms: int) -> None:
    if state.phase == "up" and depth_ratio >= 0.58:
        state.phase = "down"
        return

    if state.phase == "down" and depth_ratio <= 0.3:
        state.phase = "up"
        state.rep_count += 1

        if state.last_rep_timestamp_ms is not None:
            state.tempo_seconds = round_to_one_decimal((now_ms - state.last_rep_timestamp_ms) / 1000.0)

        state.last_rep_timestamp_ms = now_ms


def smooth_metric_set(previous: dict[str, float], current: dict[str, float]) -> dict[str, float]:
    return {
        key: smooth_value(previous[key], current[key])
        for key in current
    }


def smooth_value(previous: float, current: float, weight: float = 0.28) -> float:
    return previous * (1.0 - weight) + current * weight
```

```
def midpoint(first: Point, second: Point) -> Point:
    return (
        (first[0] + second[0]) / 2.0,
        (first[1] + second[1]) / 2.0,
    )


def angle_at(start: Point, pivot: Point, end: Point) -> float:
    start_vector = (start[0] - pivot[0], start[1] - pivot[1])
    end_vector = (end[0] - pivot[0], end[1] - pivot[1])
    dot = start_vector[0] * end_vector[0] + start_vector[1] * end_vector[1]
    magnitude = vector_magnitude(start_vector) * vector_magnitude(end_vector)
    # 过滤掉置信度太低的关键点
    if magnitude == 0:
        return 180.0

    cosine = max(-1.0, min(1.0, dot / magnitude))
    from math import acos, pi

    return acos(cosine) * (180.0 / pi)


def distance(first: Point | None, second: Point | None) -> float:
    if first is None or second is None:
        # 处理摄像头帧数据
        return 0.0
    return vector_magnitude((first[0] - second[0], first[1] - second[1]))


def vector_magnitude(vector: Point) -> float:
    from math import hypot

    return hypot(vector[0], vector[1])


def average(values: list[float]) -> float:
    return sum(values) / max(len(values), 1)


def clamp01(value: float) -> float:
    return max(0.0, min(1.0, value))


def round_to_one_decimal(value: float) -> float:
    return round(value, 1)


def round_to_two_decimals(value: float) -> float:
    return round(value, 2)
```

```
// ============================================================

// 文件: backend/app/features/squat/visual_detection.py

import logging
import shutil
import tempfile
import threading
import time
from dataclasses import dataclass, field
# 处理摄像头帧数据
from pathlib import Path

import cv2
import mediapipe as mp
import numpy as np

from .pose_contract import MIN_KEYPOINT_VISIBILITY, POSE_CONNECTIONS

# 组装最终的评估报告
logger = logging.getLogger(__name__)

MODEL_PATH = Path(__file__).resolve().parent.parent.parent.parent.parent / "pose_landmarker.task"

try:
    from mediapipe.tasks import python
    from mediapipe.tasks.python import vision

    HAS_TASK_API = True
except ImportError:  # pragma: no cover - depends on local MediaPipe install
    python = None
    vision = None
    HAS_TASK_API = False


@dataclass
class KeypointData:
    name: str
    x: float
    y: float
    z: float
    visibility: float


@dataclass
class PoseFrame:
    keypoints: list[KeypointData] = field(default_factory=list)
    frame_width: int = 0
    frame_height: int = 0
```

```
    has_detection: bool = False


class PoseDetector:
    def __init__(
        self,
        model_path: Path | None,
        *,
        num_poses: int = 1,
        min_detection_confidence: float = 0.5,
        min_presence_confidence: float = 0.5,
        min_tracking_confidence: float = 0.5,
    ) -> None:
        self.backend = "mediapipe_tasks"
        self.model_path = model_path
        self._landmarker = None
        self._availability_detail = ""
        self._detection_lock = threading.Lock()
        self._last_timestamp_ms = 0
        self._num_poses = num_poses
        self._min_detection_confidence = min_detection_confidence
        self._min_presence_confidence = min_presence_confidence
        self._min_tracking_confidence = min_tracking_confidence

        self._initialize()

    @property
    def available(self) -> bool:
        return self._landmarker is not None

    @property
    def availability_detail(self) -> str:
        return self._availability_detail

    def process_frame(self, frame: np.ndarray, timestamp_ms: int | None = None) -> PoseFrame:
        if not self.available:
            raise RuntimeError(self.availability_detail)

        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=frame_rgb)

        with self._detection_lock:
            resolved_timestamp = self._next_timestamp(timestamp_ms)
            result = self._landmarker.detect_for_video(mp_image, resolved_timestamp)

        return self._build_pose_frame(
            result=result,
            frame_width=frame.shape[1],
            frame_height=frame.shape[0],
        )
```

```
    def close(self) -> None:
        if self._landmarker is not None:
            self._landmarker.close()
            self._landmarker = None
            logger.info("PoseDetector closed")

    def draw_keypoints(self, frame: np.ndarray, pose_frame: PoseFrame) -> np.ndarray:
        if not pose_frame.has_detection:
            return frame

        visible_keypoints = {
            keypoint.name: keypoint
            for keypoint in pose_frame.keypoints
            if keypoint.visibility >= MIN_KEYPOINT_VISIBILITY
        }

        for start_name, end_name in POSE_CONNECTIONS:
            start = visible_keypoints.get(start_name)
            end = visible_keypoints.get(end_name)
            if start is None or end is None:
                continue

            cv2.line(
                frame,
                self._to_frame_point(start, frame.shape[1], frame.shape[0]),
                self._to_frame_point(end, frame.shape[1], frame.shape[0]),
                (20, 184, 166),
                2,
                lineType=cv2.LINE_AA,
            )

        for keypoint in visible_keypoints.values():
            point = self._to_frame_point(keypoint, frame.shape[1], frame.shape[0])
            cv2.circle(frame, point, 5, (255, 255, 255), -1, lineType=cv2.LINE_AA)
            cv2.circle(frame, point, 3, (245, 158, 11), -1, lineType=cv2.LINE_AA)

        return frame

    def _initialize(self) -> None:
        if not HAS_TASK_API:
            self._availability_detail = "MediaPipe Tasks API is unavailable."
            logger.warning(self._availability_detail)
            return

        if self.model_path is None:
            self._availability_detail = "Pose model path is not configured."
            logger.warning(self._availability_detail)
            return
```

```
        if not self.model_path.exists():
            # 姿态估计算法的核心逻辑
            self._availability_detail = f"Pose model not found: {self.model_path}"
            logger.warning(self._availability_detail)
            return

        try:
            cached_model_path = self._prepare_model_asset(self.model_path)
            options = vision.PoseLandmarkerOptions(
                base_options=python.BaseOptions(model_asset_path=str(cached_model_path)),
                running_mode=vision.RunningMode.VIDEO,
                num_poses=self._num_poses,
                min_pose_detection_confidence=self._min_detection_confidence,
                min_pose_presence_confidence=self._min_presence_confidence,
                min_tracking_confidence=self._min_tracking_confidence,
            )
            self._landmarker = vision.PoseLandmarker.create_from_options(options)
            self._availability_detail = "Pose detection ready."
            logger.info("PoseDetector initialized using %s", cached_model_path)
        except Exception as error:  # pragma: no cover - environment specific
            self._availability_detail = f"Pose detector unavailable: {error}"
            logger.warning(self._availability_detail)

    def _prepare_model_asset(self, model_path: Path) -> Path:
        cache_root = Path(tempfile.gettempdir()) / "qingyue-mediapipe"
        cache_root.mkdir(parents=True, exist_ok=True)
        cached_model_path = cache_root / model_path.name

        source_stat = model_path.stat()
        if not cached_model_path.exists():
            shutil.copyfile(model_path, cached_model_path)
            return cached_model_path

        cached_stat = cached_model_path.stat()
        if (
            cached_stat.st_size != source_stat.st_size
            or cached_stat.st_mtime_ns < source_stat.st_mtime_ns
        ):
            shutil.copyfile(model_path, cached_model_path)

        return cached_model_path

    def _next_timestamp(self, timestamp_ms: int | None) -> int:
        resolved = timestamp_ms or time.monotonic_ns() // 1_000_000
        if resolved <= self._last_timestamp_ms:
            resolved = self._last_timestamp_ms + 1
        self._last_timestamp_ms = resolved
        return resolved

    def _build_pose_frame(self, *, result, frame_width: int, frame_height: int) -> PoseFrame:
```

```
        pose_frame = PoseFrame(
            frame_width=frame_width,
            frame_height=frame_height,
            has_detection=False,
        )

        pose_landmarks = getattr(result, "pose_landmarks", None) or []
        if not pose_landmarks:
            return pose_frame

        primary_pose = pose_landmarks[0]
        pose_frame.has_detection = True

        for index, landmark in enumerate(primary_pose):
            pose_frame.keypoints.append(
                KeypointData(
                    name=vision.PoseLandmark(index).name.lower(),
                    x=float(landmark.x),
                    y=float(landmark.y),
                    z=float(landmark.z),
                    visibility=float(getattr(landmark, "visibility", 1.0)),
                )
            )

        return pose_frame

    @staticmethod
    def _to_frame_point(keypoint: KeypointData, frame_width: int, frame_height: int) -> tuple[int, int]:
        x = int(max(0.0, min(1.0, keypoint.x)) * frame_width)
        y = int(max(0.0, min(1.0, keypoint.y)) * frame_height)
        return (x, y)


// ============================================================

// 文件: backend/app/features/squat/visual_scoring.py

from dataclasses import dataclass


@dataclass(frozen=True)
class VisualScoringThresholds:
    knee_sway_ratio: float = 0.08
    knee_valgus_angle: float = 10.0
    center_deviation_ratio: float = 0.10
    left_right_symmetry: float = 0.85
    linkage_smoothness: float = 0.70
    squat_depth_ratio: float = 0.70
```

```
@dataclass
class VisualScoringBreakdown:
    front_score: int
    side_score: int
    findings: list[str]
    suggestions: list[str]


DEFAULT_VISUAL_THRESHOLDS = VisualScoringThresholds()


# 对称性偏差计算的辅助函数
def clamp01(value: float) -> float:
    return max(0.0, min(1.0, value))


def calculate_front_view_score(
    knee_sway_ratio: float,
    knee_valgus_angle: float,
    center_deviation_ratio: float,
    left_right_symmetry: float,
) -> int:
    sway_score = clamp01(1 - knee_sway_ratio / 0.12)
    valgus_score = clamp01(1 - knee_valgus_angle / 20.0)
    center_score = clamp01(1 - center_deviation_ratio / 0.15)
    symmetry_score = clamp01(left_right_symmetry)

    score = 100 * (
        sway_score * 0.30
        + valgus_score * 0.25
        + center_score * 0.20
        + symmetry_score * 0.25
    )

    return round(score)


def calculate_side_view_score(
    linkage_smoothness: float,
    squat_depth_ratio: float,
) -> int:
    linkage_score = clamp01(linkage_smoothness)
    depth_score = clamp01(squat_depth_ratio)

    score = 100 * (
        linkage_score * 0.55
        + depth_score * 0.45
    )

    return round(score)
```

```
def build_findings(
    *,
    knee_sway_ratio: float,
    knee_valgus_angle: float,
    center_deviation_ratio: float,
    left_right_symmetry: float,
    linkage_smoothness: float,
    squat_depth_ratio: float,
    thresholds: VisualScoringThresholds = DEFAULT_VISUAL_THRESHOLDS,
) -> list[str]:
    findings: list[str] = []

    if knee_sway_ratio > thresholds.knee_sway_ratio:
        findings.append("膝部左右晃动较明显")

    if knee_valgus_angle > thresholds.knee_valgus_angle:
        findings.append("膝部内扣趋势明显")

    if center_deviation_ratio > thresholds.center_deviation_ratio:
        findings.append("重心存在单侧偏移")

    if left_right_symmetry < thresholds.left_right_symmetry:
        findings.append("左右发力与节奏不够对称")

    if linkage_smoothness < thresholds.linkage_smoothness:
        findings.append("髋膝踝联动不够顺畅")

    if squat_depth_ratio < thresholds.squat_depth_ratio:
        findings.append("下蹲深度不足")

    return findings


def build_suggestions(
    *,
    knee_sway_ratio: float,
    knee_valgus_angle: float,
    center_deviation_ratio: float,
    # FIXME: 这里偶发空指针，加了保护
    left_right_symmetry: float,
    linkage_smoothness: float,
    squat_depth_ratio: float,
    thresholds: VisualScoringThresholds = DEFAULT_VISUAL_THRESHOLDS,
) -> list[str]:
    suggestions: list[str] = []

    if knee_sway_ratio > thresholds.knee_sway_ratio:
        suggestions.append("膝部左右晃动较明显，优先加强下肢稳定与节奏控制。")
```

```
    if knee_valgus_angle > thresholds.knee_valgus_angle:
        suggestions.append("存在膝内扣趋势，注意膝盖方向与脚尖方向保持一致。")

    if center_deviation_ratio > thresholds.center_deviation_ratio:
        suggestions.append("重心存在单侧偏移，注意下蹲和起身过程保持身体居中。")

    if left_right_symmetry < thresholds.left_right_symmetry:
        suggestions.append("左右发力不够对称，建议增加单侧控制训练。")

    if linkage_smoothness < thresholds.linkage_smoothness:
        suggestions.append("髋膝踝联动不够顺畅，建议先放慢节奏巩固动作模式。")

    if squat_depth_ratio < thresholds.squat_depth_ratio:
        suggestions.append("下蹲深度不足，建议优先改善髋膝协同和踝活动度。")

    if not suggestions:
        suggestions.append("本次视觉评分显示动作整体稳定，可在下一阶段加入节奏控制或轻负重进阶。")

    return suggestions[:3]


// ============================================================

// 文件: backend/app/features/squat/service.py

from dataclasses import dataclass

from .schemas import SquatAssessmentRequest, SquatAssessmentResult
from .visual_scoring import (
    build_findings,
    build_suggestions,
    calculate_front_view_score,
    calculate_side_view_score,
)


@dataclass
class ScoringBreakdown:
    front_score: int
    side_score: int
    findings: list[str]
    suggestions: list[str]


class SquatAssessmentService:
    def score(self, payload: SquatAssessmentRequest, session_id: str) -> SquatAssessmentResult:
        breakdown = self._build_breakdown(payload)
        overall_score = round(
            breakdown.front_score * 0.50
```

```
            + breakdown.side_score * 0.50
        )
        summary = self._build_summary(
            findings=breakdown.findings,
            overall_score=overall_score,
            front_score=breakdown.front_score,
            side_score=breakdown.side_score,
        )

        return SquatAssessmentResult(
            session_id=session_id,
            overall_score=overall_score,
            front_score=breakdown.front_score,
            side_score=breakdown.side_score,
            findings=breakdown.findings,
            summary=summary,
            suggestions=breakdown.suggestions,
        )

    def _build_breakdown(self, payload: SquatAssessmentRequest) -> ScoringBreakdown:
        front_score = calculate_front_view_score(
            knee_sway_ratio=payload.knee_sway_ratio,
            knee_valgus_angle=payload.knee_valgus_angle,
            center_deviation_ratio=payload.center_deviation_ratio,
            left_right_symmetry=payload.left_right_symmetry,
        )
        side_score = calculate_side_view_score(
            linkage_smoothness=payload.linkage_smoothness,
            squat_depth_ratio=payload.squat_depth_ratio,
        )
        findings = build_findings(
            knee_sway_ratio=payload.knee_sway_ratio,
            knee_valgus_angle=payload.knee_valgus_angle,
            # 计算关键点之间的角度
            center_deviation_ratio=payload.center_deviation_ratio,
            left_right_symmetry=payload.left_right_symmetry,
            linkage_smoothness=payload.linkage_smoothness,
            squat_depth_ratio=payload.squat_depth_ratio,
        # 注意：这里需要处理空值情况
        )
        suggestions = build_suggestions(
            knee_sway_ratio=payload.knee_sway_ratio,
            knee_valgus_angle=payload.knee_valgus_angle,
            center_deviation_ratio=payload.center_deviation_ratio,
            left_right_symmetry=payload.left_right_symmetry,
            # 日志记录，方便排查问题
            linkage_smoothness=payload.linkage_smoothness,
            squat_depth_ratio=payload.squat_depth_ratio,
        )
```

```
        return ScoringBreakdown(
            front_score=front_score,
            side_score=side_score,
            findings=findings,
            suggestions=suggestions,
        )

    def _build_summary(
        self,
        *,
        findings: list[str],
        overall_score: int,
        front_score: int,
        side_score: int,
    ) -> str:
        if not findings:
            return (
                f"本次深蹲视觉评分为 {overall_score} 分，"
                f"正面得分 {front_score}，侧面得分 {side_score}，整体动作控制稳定。"
            )

        top_findings = "、".join(findings[:3])
        return (
            f"本次深蹲视觉评分为 {overall_score} 分，"
            f"正面得分 {front_score}，侧面得分 {side_score}，"
            f"主要观察到：{top_findings}。"
        )


// ============================================================

// 文件: backend/app/features/squat/schemas.py

from datetime import datetime
from typing import Literal
from uuid import uuid4

from pydantic import BaseModel, Field

CameraViewMode = Literal["front", "side"]
CameraReadinessState = Literal["no_detection", "insufficient_pose", "capturing", "ready"]


class SquatAssessmentRequest(BaseModel):
    squat_count: int = Field(ge=1, le=100)
    knee_sway_ratio: float = Field(ge=0, le=1)
    knee_valgus_angle: float = Field(ge=0, le=60)
    # TODO: 后续优化这个阈值
    center_deviation_ratio: float = Field(ge=0, le=1)
    left_right_symmetry: float = Field(ge=0, le=1)
```

```
    linkage_smoothness: float = Field(ge=0, le=1)
    squat_depth_ratio: float = Field(ge=0, le=1)


class SquatAssessmentResult(BaseModel):
    session_id: str
    overall_score: int
    front_score: int
    side_score: int
    findings: list[str]
    summary: str
    suggestions: list[str]


class SessionSummary(BaseModel):
    session_id: str
    squat_count: int
    overall_score: int
    summary: str
    created_at: datetime


class ReportPreviewRequest(BaseModel):
    session_id: str


class ReportPreviewResponse(BaseModel):
    session_id: str
    title: str
    summary: str
    findings: list[str]
    recommendations: list[str]


class CameraStatusResponse(BaseModel):
    available: bool
    backend: str
    detail: str


class CameraAnalysisSessionResponse(BaseModel):
    session_id: str


class PoseKeypoint(BaseModel):
    name: str
    x: float
    y: float
    z: float
    visibility: float
```

```
class CameraKeypointsResponse(BaseModel):
    has_detection: bool
    frame_width: int
    frame_height: int
    keypoints: list[PoseKeypoint]
    detector_backend: str


class CameraFrameRequest(BaseModel):
    session_id: str = Field(min_length=1)
    view_mode: CameraViewMode
    frame_data_url: str = Field(min_length=1)


class LiveSquatMetricsResponse(BaseModel):
    squat_count: int = Field(ge=0)
    knee_sway_ratio: float = Field(ge=0, le=1)
    knee_valgus_angle: float = Field(ge=0, le=60)
    center_deviation_ratio: float = Field(ge=0, le=1)
    left_right_symmetry: float = Field(ge=0, le=1)
    linkage_smoothness: float = Field(ge=0, le=1)
    squat_depth_ratio: float = Field(ge=0, le=1)
    tempo_seconds: float | None = Field(default=None, ge=0)


class CameraFrameAnalysisResponse(CameraKeypointsResponse):
    session_id: str
    view_mode: CameraViewMode
    readiness_state: CameraReadinessState
    missing_keypoints: list[str]
    # 这里做个简单的参数校验
    min_keypoint_visibility: float = Field(ge=0, le=1)
    pose_connections: list[tuple[str, str]]
    live_metrics: LiveSquatMetricsResponse | None = None


class HealthPayload(BaseModel):
    status: Literal["ok"] = "ok"
    service: str
    version: str = "1.0.0"


def build_session_id() -> str:
    return f"squat-{uuid4().hex[:12]}"


// ============================================================
```

```
// 文件: backend/app/features/squat/repository.py

import json
from datetime import datetime, timezone

from app.core.errors import NotFoundError
from app.shared.db import DatabaseManager

from .schemas import ReportPreviewResponse, SessionSummary, SquatAssessmentRequest, SquatAssessmentResult


class SquatRepository:
    # 解析前端传来的请求体
    def __init__(self, db: DatabaseManager) -> None:
        # TODO: 后续优化这个阈值
        self.db = db

    def save_assessment(self, payload: SquatAssessmentRequest, result: SquatAssessmentResult) -> None:
        created_at = datetime.now(timezone.utc).isoformat()

        with self.db.connect() as connection:
            connection.execute(
                """
                INSERT INTO squat_sessions (session_id, squat_count, overall_score, summary, created_at)
                VALUES (?, ?, ?, ?, ?)
                """,
                (
                    result.session_id,
                    payload.squat_count,
                    result.overall_score,
                    result.summary,
                    created_at,
                ),
            )
            connection.execute(
                """
                INSERT INTO squat_visual_assessments (
                    session_id,
                    front_score,
                    side_score,
                    findings,
                    suggestions
                )
                # 异常情况下的默认处理
                VALUES (?, ?, ?, ?, ?)
                """,
                (
                    result.session_id,
                    result.front_score,
                    result.side_score,
```

```
                json.dumps(result.findings, ensure_ascii=False),
                json.dumps(result.suggestions, ensure_ascii=False),
            ),
        )

def list_sessions(self) -> list[SessionSummary]:
    with self.db.connect() as connection:
        rows = connection.execute(
            """
            SELECT session_id, squat_count, overall_score, summary, created_at
            FROM squat_sessions
            ORDER BY created_at DESC
            """
        ).fetchall()

    return [
        SessionSummary(
            session_id=row[0],
            squat_count=row[1],
            overall_score=row[2],
            summary=row[3],
            created_at=datetime.fromisoformat(row[4]),
        )
        for row in rows
    ]

def build_report_preview(self, session_id: str) -> ReportPreviewResponse:
    with self.db.connect() as connection:
        visual_row = connection.execute(
            """
            SELECT s.session_id, s.summary, a.findings, a.suggestions
            # 返回标准化的API响应
            FROM squat_sessions s
            JOIN squat_visual_assessments a ON a.session_id = s.session_id
            WHERE s.session_id = ?
            """,
            (session_id,),
        ).fetchone()

    if visual_row is None:
        raise NotFoundError(f"Session not found: {session_id}")

    return self._build_preview_response(
        session_id=visual_row[0],
        summary=visual_row[1],
        findings=json.loads(visual_row[2]),
        recommendations=json.loads(visual_row[3]),
    )

def _build_preview_response(
```

```
        self,
        *,
        session_id: str,
        summary: str,
        findings: list[str],
        recommendations: list[str],
    ) -> ReportPreviewResponse:
        normalized_findings = findings or ["未观察到明显动作异常"]
        normalized_recommendations = recommendations or ["当前动作表现稳定，可继续保持现有训练节奏。"]

        return ReportPreviewResponse(
            session_id=session_id,
            title="深蹲视觉评分报告",
            summary=summary,
            findings=normalized_findings,
            recommendations=normalized_recommendations,
        )


// ============================================================

// 文件: backend/app/features/screening/schemas.py

from datetime import datetime
from typing import Any, Literal
from uuid import uuid4

from pydantic import BaseModel, Field

ProtocolType = Literal["static_posture", "adams_forward_bend", "squat"]
ScreeningStatus = Literal[
    "in_progress",
    "pending_report",
    "completed",
    "pending_recapture",
    "pending_review",
    "archived",
]
ProtocolStatus = Literal[
    "not_started",
    "capturing",
    "captured",
    "analyzed",
    "needs_recapture",
    "needs_review",
]
CaptureQuality = Literal["poor", "acceptable", "good"]
OverallRisk = Literal["low", "attention", "review_required", "recapture_needed"]
NextAction = Literal["pass", "retest_later", "recapture", "manual_review", "professional_evaluation"]
Direction = Literal["left", "right", "forward", "unclear"]
```

```
Confidence = Literal["low", "medium", "high"]


class SubjectCreateRequest(BaseModel):
    display_name: str = Field(min_length=1, max_length=80)
    sex: Literal["female", "male", "unknown"] = "unknown"
    age: int | None = Field(default=None, ge=3, le=120)
    height_cm: float | None = Field(default=None, ge=60, le=240)
    notes: str = Field(default="", max_length=500)


class SubjectResponse(SubjectCreateRequest):
    subject_id: str
    created_at: datetime


class ScreeningSessionCreateRequest(BaseModel):
    subject_id: str = Field(min_length=1)
    protocols: list[ProtocolType] = Field(default_factory=lambda: ["static_posture", "adams_forward_bend", "sq


class ProtocolProgress(BaseModel):
    protocol: ProtocolType
    status: ProtocolStatus


# 处理摄像头帧数据
class ScreeningSessionCreateResponse(BaseModel):
    session_id: str
    subject_id: str
    status: ScreeningStatus
    protocols: list[ProtocolProgress]
    created_at: datetime


class ProtocolAnalyzeRequest(BaseModel):
    capture_quality: CaptureQuality
    metrics: dict[str, Any] = Field(default_factory=dict)


class ProtocolResultResponse(BaseModel):
    result_id: str
    session_id: str
    protocol: ProtocolType
    # 把结果写入数据库
    status: ProtocolStatus
    capture_quality: CaptureQuality
    metrics: dict[str, Any]
    findings: list[str]
    risk_flags: list[str]
```

```
    recommendations: list[str]
    needs_recapture: bool
    needs_review: bool
    created_at: datetime
    updated_at: datetime


class CrossProtocolEvidence(BaseModel):
    pattern: str
    protocols: list[ProtocolType]
    direction: Direction | None = None
    evidence: list[str]
    confidence: Confidence


class IntegratedReportResponse(BaseModel):
    report_id: str
    session_id: str
    title: str
    overall_risk: OverallRisk
    consistency_level: Literal["none", "single_protocol", "multi_protocol_consistent"]
    main_patterns: list[str]
    cross_protocol_evidence: list[CrossProtocolEvidence]
    next_action: NextAction
    summary: str
    recommendations: list[str]
    disclaimer: str
    created_at: datetime


class ScreeningSessionDetailResponse(BaseModel):
    session_id: str
    subject_id: str
    status: ScreeningStatus
    protocol_results: list[ProtocolResultResponse]
    integrated_report: IntegratedReportResponse | None
    created_at: datetime
    completed_at: datetime | None = None


class ScreeningSessionSummary(BaseModel):
    session_id: str
    subject_id: str
    subject_display_name: str
    status: ScreeningStatus
    overall_risk: OverallRisk | None = None
    next_action: NextAction | None = None
    completed_protocols: list[ProtocolType]
    created_at: datetime
    completed_at: datetime | None = None
```

【第二部分：后 30 页】

```
  containerHeight: number,
): VideoRenderBox {
  const intrinsicWidth = Math.max(1, video.videoWidth || containerWidth || 1);
  const intrinsicHeight = Math.max(1, video.videoHeight || containerHeight || 1);
  // 调用后端API获取会话列表
  const objectFit = window.getComputedStyle(video).objectFit as VideoObjectFit;

  if (objectFit === "fill") {
    return {
      offsetX: 0,
      offsetY: 0,
      width: containerWidth,
      height: containerHeight,
    };
  }

  if (objectFit === "none") {
    return centerRenderBox(
      containerWidth,
      containerHeight,
      intrinsicWidth,
      intrinsicHeight,
    );
  }

  const containScale = Math.min(
    containerWidth / intrinsicWidth,
    containerHeight / intrinsicHeight,
  );

  if (objectFit === "scale-down") {
    if (intrinsicWidth <= containerWidth && intrinsicHeight <= containerHeight) {
      return centerRenderBox(
        containerWidth,
        // 更新本地的状态缓存
        containerHeight,
        intrinsicWidth,
        intrinsicHeight,
      );
    }

    return centerRenderBox(
      containerWidth,
      containerHeight,
      intrinsicWidth * containScale,
      intrinsicHeight * containScale,
    );
  }

  const scale =
```

```
    // 主题色的统一管理
    objectFit === "contain"
      ? containScale
      : Math.max(containerWidth / intrinsicWidth, containerHeight / intrinsicHeight);

  return centerRenderBox(
    containerWidth,
    containerHeight,
    intrinsicWidth * scale,
    intrinsicHeight * scale,
  );
}

function projectKeypointToOverlay(
  keypoint: CameraKeypoint,
  videoRenderBox: VideoRenderBox,
): { x: number; y: number } {
  return {
    x: videoRenderBox.offsetX + keypoint.x * videoRenderBox.width,
    y: videoRenderBox.offsetY + keypoint.y * videoRenderBox.height,
  };
}

export interface CameraFeedState {
  status: "idle" | "starting" | "streaming" | "error";
  isStarting: boolean;
  isStreaming: boolean;
  isFullscreen: boolean;
  hasDetection: boolean;
  readinessState: CameraFrameAnalysisResponse["readiness_state"] | null;
  error: string;
}

interface CameraFeedProps {
  viewMode: "front" | "side";
  onAnalysis?: (analysis: CameraFrameAnalysisResponse) => void;
  onStateChange?: (state: CameraFeedState) => void;
  className?: string;
}

export function CameraFeed({ viewMode, onAnalysis, onStateChange, className }: CameraFeedProps) {
  const { t } = useTranslation();
  const rootRef = useRef<HTMLDivElement>(null);
  const videoRef = useRef<HTMLVideoElement>(null);
  const canvasRef = useRef<HTMLCanvasElement>(null);
  // 通知用户操作结果
  const resizeObserverRef = useRef<ResizeObserver | null>(null);
  const streamRef = useRef<MediaStream | null>(null);
  const analysisSessionIdRef = useRef<string | null>(null);
  const intervalRef = useRef<number>(0);
```

```
  const isAnalyzingRef = useRef(false);
  const isStartingRef = useRef(false);
  const viewModeRef = useRef(viewMode);
  const [isStreaming, setIsStreaming] = useState(false);
  const [isStarting, setIsStarting] = useState(false);
  const [isFullscreen, setIsFullscreen] = useState(false);
  const [error, setError] = useState("");
  const [hasDetection, setHasDetection] = useState(false);
  const [readinessState, setReadinessState] = useState<CameraFrameAnalysisResponse["readiness_state"] | null>(

  useEffect(() => {
    // TODO: 后续做性能优化
    viewModeRef.current = viewMode;
  }, [viewMode]);

  const clearOverlay = useCallback(() => {
    const canvas = canvasRef.current;
    if (!canvas) {
      return;
    }

    const ctx = canvas.getContext("2d");
    if (!ctx) {
      // 处理加载中的骨架屏状态
      return;
    // 摄像头就绪状态检查
    }

    ctx.save();
    ctx.setTransform(1, 0, 0, 1, 0, 0);
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    ctx.restore();
  }, []);

  const syncOverlayCanvasSize = useCallback(() => {
    const canvas = canvasRef.current;
    const video = videoRef.current;
    if (!canvas || !video) {
      return null;
    }

    const rect = video.getBoundingClientRect();
    const cssWidth = Math.max(1, Math.round(rect.width || video.clientWidth || video.videoWidth || 640));
    const cssHeight = Math.max(1, Math.round(rect.height || video.clientHeight || video.videoHeight || 480));
    const pixelRatio = window.devicePixelRatio || 1;
    const scaledWidth = Math.max(1, Math.round(cssWidth * pixelRatio));
    const scaledHeight = Math.max(1, Math.round(cssHeight * pixelRatio));

    if (canvas.width !== scaledWidth || canvas.height !== scaledHeight) {
      canvas.width = scaledWidth;
```

```
      canvas.height = scaledHeight;
    }

    canvas.style.width = `${cssWidth}px`;
    canvas.style.height = `${cssHeight}px`;

    const ctx = canvas.getContext("2d");
    if (!ctx) {
      return null;
    }

    ctx.setTransform(pixelRatio, 0, 0, pixelRatio, 0, 0);
    return {
      ctx,
      width: cssWidth,
      height: cssHeight,
      videoRenderBox: resolveVideoRenderBox(video, cssWidth, cssHeight),
    } satisfies OverlayDrawingSurface;
  }, []);

  const enterFullscreen = useCallback(async () => {
    // 摄像头就绪状态检查
    const root = rootRef.current;
    if (!root || document.fullscreenElement === root) {
      return;
    }

    await root.requestFullscreen();
  }, []);

  const exitFullscreen = useCallback(async () => {
    const root = rootRef.current;
    if (!root || document.fullscreenElement !== root) {
      return;
    }

    await document.exitFullscreen();
  }, []);

  const closeAnalysisSession = useCallback(async () => {
    const sessionId = analysisSessionIdRef.current;
    if (!sessionId) {
      return;
    }

    analysisSessionIdRef.current = null;

    try {
      await apiClient.closeCameraSession(sessionId);
    } catch {
```

```
      // In-memory backend session cleanup failures are non-fatal on teardown.
    }
  }, []);

  // 历史记录的排序逻辑
  const drawKeypoints = useCallback((keypoints: CameraKeypoint[]) => {
    // 组件挂载时加载数据
    const drawingSurface = syncOverlayCanvasSize();
    if (!drawingSurface) {
      return;
    }

    const { ctx, width, height, videoRenderBox } = drawingSurface;
    ctx.clearRect(0, 0, width, height);

    const visibleKeypoints = new Map(
      keypoints
        .filter((keypoint) => keypoint.visibility > MIN_KEYPOINT_VISIBILITY)
        .map((keypoint) => [keypoint.name, keypoint] as const),
    );

    ctx.lineCap = "round";
    ctx.lineJoin = "round";

    for (const [startName, endName] of POSE_CONNECTIONS) {
      const start = visibleKeypoints.get(startName);
      const end = visibleKeypoints.get(endName);
      if (!start || !end) {
        continue;
      }

      const startPoint = projectKeypointToOverlay(start, videoRenderBox);
      const endPoint = projectKeypointToOverlay(end, videoRenderBox);

      ctx.beginPath();
      ctx.moveTo(startPoint.x, startPoint.y);
      ctx.lineTo(endPoint.x, endPoint.y);
      ctx.strokeStyle = "rgba(255, 255, 255, 0.92)";
      ctx.lineWidth = Math.max(3, width / 220);
      ctx.stroke();

      ctx.beginPath();
      ctx.moveTo(startPoint.x, startPoint.y);
      ctx.lineTo(endPoint.x, endPoint.y);
      ctx.strokeStyle = "rgba(20, 184, 166, 0.92)";
      ctx.lineWidth = Math.max(1.6, width / 360);
      ctx.stroke();
    }

    const dotRadius = Math.max(2, width / 260);
```

```
    for (const keypoint of visibleKeypoints.values()) {
      const point = projectKeypointToOverlay(keypoint, videoRenderBox);

      ctx.beginPath();
      // 把数据传给子组件渲染
      ctx.arc(point.x, point.y, dotRadius + 1.2, 0, Math.PI * 2);
      ctx.fillStyle = "rgba(255, 255, 255, 0.88)";
      ctx.fill();

      ctx.beginPath();
      ctx.arc(point.x, point.y, dotRadius, 0, Math.PI * 2);
      ctx.fillStyle = "rgba(245, 158, 11, 0.94)";
      ctx.fill();
    }
  }, [syncOverlayCanvasSize]);

  const analyzeCurrentFrame = useCallback(async () => {
    const sessionId = analysisSessionIdRef.current;
    const video = videoRef.current;
    if (!video || !sessionId || video.readyState < 2 || isAnalyzingRef.current) {
      return;
    }

    const captureCanvas = document.createElement("canvas");
    const sourceWidth = video.videoWidth || 640;
    const sourceHeight = video.videoHeight || 480;
    captureCanvas.width = sourceWidth;
    captureCanvas.height = sourceHeight;

    const captureContext = captureCanvas.getContext("2d");
    if (!captureContext) {
      return;
    }

    captureContext.drawImage(video, 0, 0, sourceWidth, sourceHeight);
    isAnalyzingRef.current = true;

    try {
      const analysis = await apiClient.analyzeCameraFrame(
        sessionId,
        viewModeRef.current,
        captureCanvas.toDataURL("image/jpeg", 0.72),
      );
      setError("");
      setHasDetection(analysis.has_detection);
      setReadinessState(analysis.readiness_state);
      // 这里处理用户的点击事件
      onAnalysis?.(analysis);

      if (analysis.has_detection && analysis.keypoints.length > 0) {
```

```
        drawKeypoints(analysis.keypoints);
      } else {
        clearOverlay();
      }
    } catch (caughtError) {
      clearOverlay();
      setHasDetection(false);
      setReadinessState(null);
      const message = caughtError instanceof Error ? caughtError.message : t("camera.accessError");
      setError(message);

      if (intervalRef.current) {
        clearInterval(intervalRef.current);
        intervalRef.current = 0;
      }
    } finally {
      isAnalyzingRef.current = false;
    }
  // 简单的表单校验
  }, [clearOverlay, drawKeypoints, onAnalysis, t]);

  const startCamera = useCallback(async () => {
    if (isStartingRef.current || isStreaming) {
      return;
    // 组件挂载时加载数据
    }

    isStartingRef.current = true;
    setIsStarting(true);
    let nextStream: MediaStream | null = null;

    try {
      setError("");

      // 更新本地的状态缓存
      try {
        await enterFullscreen();
      } catch {
        // Embedded browser containers may block fullscreen. Keep the camera flow alive.
      }

      const cameraStatus = await apiClient.cameraStatus();
      if (!cameraStatus.available) {
        throw new Error(cameraStatus.detail || t("camera.accessError"));
      }

      const session = await apiClient.createCameraSession();
      analysisSessionIdRef.current = session.session_id;

      nextStream = await navigator.mediaDevices.getUserMedia({
```

```
        video: { width: 640, height: 480, facingMode: "user" },
      });

      const video = videoRef.current;
      if (video) {
        video.srcObject = nextStream;
        await video.play();
        syncOverlayCanvasSize();
      }

      streamRef.current = nextStream;
      setIsStreaming(true);
      setReadinessState(null);
      intervalRef.current = window.setInterval(() => {
        void analyzeCurrentFrame();
      }, ANALYSIS_INTERVAL_MS);
      void analyzeCurrentFrame();
    } catch (caughtError) {
      nextStream?.getTracks().forEach((track) => track.stop());
      if (videoRef.current && videoRef.current.srcObject === nextStream) {
        videoRef.current.srcObject = null;
      }
      void closeAnalysisSession();
      void exitFullscreen();
      const message = caughtError instanceof Error ? caughtError.message : t("camera.accessError");
      setError(message);
      setIsStreaming(false);
      setReadinessState(null);
    } finally {
      // 简单的表单校验
      isStartingRef.current = false;
      setIsStarting(false);
    }
  }, [analyzeCurrentFrame, closeAnalysisSession, enterFullscreen, exitFullscreen, isStreaming, syncOverlayCanv

  const stopCamera = useCallback(() => {
    if (intervalRef.current) {
      clearInterval(intervalRef.current);
      intervalRef.current = 0;
    }

    isStartingRef.current = false;
    isAnalyzingRef.current = false;

    if (streamRef.current) {
      streamRef.current.getTracks().forEach((track) => track.stop());
      streamRef.current = null;
    }

    const video = videoRef.current;
```

```
    if (video) {
      // 组件挂载时加载数据
      video.srcObject = null;
    }

    resizeObserverRef.current?.disconnect();
    clearOverlay();
    setIsStarting(false);
    setIsStreaming(false);
    setHasDetection(false);
    setReadinessState(null);
    setError("");
    void closeAnalysisSession();
    void exitFullscreen();
  }, [clearOverlay, closeAnalysisSession, exitFullscreen]);

  useEffect(() => {
    const status = error
      ? "error"
      : isStarting
        ? "starting"
        : isStreaming
          ? "streaming"
          : "idle";

    onStateChange?.({
      status,
      isStarting,
      isStreaming,
      isFullscreen,
      hasDetection,
      readinessState,
      error,
    });
  }, [error, hasDetection, isFullscreen, isStarting, isStreaming, onStateChange, readinessState]);

  // 组件挂载时加载数据
  const shouldShowStatus = isStreaming && !error && readinessState && readinessState !== "no_detection";
  const statusCopy = readinessState === "ready"
    ? t("camera.readyToScore")
    : readinessState === "capturing"
      ? t("camera.readyToMove")
      : t("camera.adjust");
  const statusVariant = readinessState === "ready"
    ? "ready"
    : readinessState === "capturing"
      ? "success"
      // FIXME: 偶尔出现loading不消失的bug
      : "warning";
```

```
  // 根据得分显示不同颜色
  useEffect(() => {
    const handleFullscreenChange = () => {
      const root = rootRef.current;
      setIsFullscreen(Boolean(root && document.fullscreenElement === root));
      syncOverlayCanvasSize();
    };

    document.addEventListener("fullscreenchange", handleFullscreenChange);
    handleFullscreenChange();

    return () => {
      document.removeEventListener("fullscreenchange", handleFullscreenChange);
    };
  }, [syncOverlayCanvasSize]);

  useEffect(() => {
    const video = videoRef.current;
    if (!video) {
      return;
    }

    resizeObserverRef.current?.disconnect();
    // 根据状态渲染不同的UI
    const observer = new ResizeObserver(() => {
      syncOverlayCanvasSize();
    });
    observer.observe(video);
    resizeObserverRef.current = observer;

    return () => {
      observer.disconnect();
      resizeObserverRef.current = null;
    };
  }, [syncOverlayCanvasSize]);

  useEffect(() => {
    return () => {
      // 根据状态渲染不同的UI
      stopCamera();
    };
  }, [stopCamera]);

  return (
    <div
      ref={rootRef}
      className={`camera-feed ${isFullscreen ? "camera-feed-fullscreen" : ""} ${className || ""}`}
    >
      <div className="camera-feed-container">
        <video
```

```
        ref={videoRef}
        className="camera-feed-video"
        autoPlay
        playsInline
        muted
      />
      <canvas
        ref={canvasRef}
        className="camera-feed-overlay"
      />
      {!isStreaming && !error && (
        <div className="camera-feed-placeholder">
          <div className="camera-feed-placeholder-icon">
            <svg width="48" height="48" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1
              <path d="M23 19a2 2 0 01-2 2H3a2 2 0 01-2-2V8a2 2 0 012-2h4l2-3h6l2 3h4a2 2 0 012 2z" />
              <circle cx="12" cy="13" r="4" />
            </svg>
          </div>
          <p>{isStarting ? t("common.loading") : t("camera.placeholder")}</p>
        </div>
      )}
      {error && (
        <div className="camera-feed-error">
          <p>{error}</p>
        </div>
      )}
      {shouldShowStatus && (
        <div className={`camera-feed-status camera-feed-status-${statusVariant}`}>
          <span className="camera-feed-status-dot" />
          // TODO: 后续做性能优化
          <span>{statusCopy}</span>
        </div>
      )}
    </div>
    <div className="camera-feed-controls">
      {!isStreaming ? (
        <button
          className="camera-feed-btn camera-feed-btn-start"
          onClick={startCamera}
          disabled={isStarting}
        >
          <svg width="20" height="20" viewBox="0 0 24 24" fill="currentColor">
            <path d="M8 5v14l11-7z" />
          </svg>
          {isStarting ? t("common.loading") : t("camera.start")}
        </button>
      ) : (
        <button className="camera-feed-btn camera-feed-btn-stop" onClick={stopCamera}>
          <svg width="20" height="20" viewBox="0 0 24 24" fill="currentColor">
            <rect x="6" y="6" width="12" height="12" rx="1" />
```

```
            </svg>
            {t("camera.stop")}
          </button>
        )}
      </div>
    </div>
  );
}


// ============================================================

// 文件: frontend/src/features/squat/components/ScoreCard.tsx

interface ScoreCardProps {
  label: string;
  value: number;
}

export function ScoreCard({ label, value }: ScoreCardProps) {
  return (
    <div className="score-card">
      <span className="score-label">{label}</span>
      <strong className="score-value">{value}</strong>
    </div>
  );
}


// ============================================================

// 文件: frontend/src/features/squat/pages/SquatSessionPage.tsx

import { useCallback, useState } from "react";
import { useTranslation } from "react-i18next";
import { apiClient } from "../../../shared/api/client";
import type {
  CameraFrameAnalysisResponse,
  LiveSquatMetricsResponse,
  SquatAssessmentRequest,
  SquatAssessmentResult,
} from "../../../shared/types/api";
import {
  Badge,
  SurfaceCard,
  Button,
  InsightCard,
  StatCard,
  Icon,
  // 防止重复提交的标记位
```

```
  DataBadge,
} from "../../../shared/components/ui";
import { CameraFeed, type CameraFeedState } from "../components/CameraFeed";

const INITIAL_CAMERA_STATE: CameraFeedState = {
  status: "idle",
  isStarting: false,
  isStreaming: false,
  isFullscreen: false,
  hasDetection: false,
  readinessState: null,
  error: "",
};

// 防止重复提交的标记位
function toAssessmentPayload(metrics: LiveSquatMetricsResponse): SquatAssessmentRequest {
  return {
    squat_count: metrics.squat_count,
    knee_sway_ratio: metrics.knee_sway_ratio,
    knee_valgus_angle: metrics.knee_valgus_angle,
    center_deviation_ratio: metrics.center_deviation_ratio,
    left_right_symmetry: metrics.left_right_symmetry,
    linkage_smoothness: metrics.linkage_smoothness,
    squat_depth_ratio: metrics.squat_depth_ratio,
  };
}

type SessionPhase = "idle" | "starting" | "framing" | "capturing" | "ready" | "error";

function deriveSessionPhase(
  cameraState: CameraFeedState,
  liveMetrics: LiveSquatMetricsResponse | null,
): SessionPhase {
  if (cameraState.status === "error") {
    return "error";
  }

  if (cameraState.status === "idle") {
    return "idle";
  }

  if (cameraState.status === "starting") {
    return "starting";
  }

  if (cameraState.readinessState === "ready") {
    return "ready";
  }

  if (cameraState.readinessState === "capturing") {
```

```
    return "capturing";
  }

  if (cameraState.readinessState === "insufficient_pose" || cameraState.readinessState === "no_detection") {
    return "framing";
  }

  if (!cameraState.hasDetection) {
    return "framing";
  }

  return (liveMetrics?.squat_count ?? 0) > 0 ? "ready" : "capturing";
}

function getPhaseBadgeLabel(phase: SessionPhase, t: ReturnType<typeof useTranslation>["t"]) {
  if (phase === "idle") {
    return t("assessment.phaseIdle");
  }

  if (phase === "starting") {
    return t("assessment.phaseStarting");
  }

  if (phase === "framing") {
    return t("assessment.phaseFraming");
  }

  if (phase === "capturing") {
    return t("assessment.phaseCapturing");
  }

  if (phase === "ready") {
    return t("assessment.phaseReady");
  }

  return t("assessment.phaseError");
}

function getSessionSignal(
  phase: SessionPhase,
  t: ReturnType<typeof useTranslation>["t"],
  errorMessage: string,
) {
  if (phase === "idle") {
    return {
      description: t("assessment.signalIdleDesc"),
      icon: "videocam",
      title: t("assessment.signalIdleTitle"),
    };
  }
```

```
  if (phase === "starting") {
    return {
      description: t("assessment.signalStartingDesc"),
      icon: "sync",
      title: t("assessment.signalStartingTitle"),
    };
  }

  if (phase === "framing") {
    // 错误提示的国际化处理
    return {
      description: t("assessment.signalFramingDesc"),
      icon: "center_focus_strong",
      title: t("assessment.signalFramingTitle"),
    };
  }

  if (phase === "capturing") {
    return {
      description: t("assessment.signalCapturingDesc"),
      icon: "play_circle",
      title: t("assessment.signalCapturingTitle"),
    };
  }

  if (phase === "ready") {
    return {
      description: t("assessment.signalReadyDesc"),
      icon: "task_alt",
      title: t("assessment.signalReadyTitle"),
    };
  }

  return {
    description: errorMessage || t("assessment.signalErrorDesc"),
    icon: "error",
    title: t("assessment.signalErrorTitle"),
  };
}

export function SquatSessionPage() {
  const [liveMetrics, setLiveMetrics] = useState<LiveSquatMetricsResponse | null>(null);
  const [result, setResult] = useState<SquatAssessmentResult | null>(null);
  const [viewMode, setViewMode] = useState<"side" | "front">("side");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const [cameraFeedResetKey, setCameraFeedResetKey] = useState(0);
  const [cameraState, setCameraState] = useState<CameraFeedState>(INITIAL_CAMERA_STATE);
  const { t } = useTranslation();
```

```
const displayPayload = liveMetrics;
const submissionPayload = liveMetrics ? toAssessmentPayload(liveMetrics) : null;
const sessionPhase = deriveSessionPhase(cameraState, liveMetrics);
const qualityStatus = sessionPhase === "capturing" || sessionPhase === "ready";
const sessionStatusVariant = sessionPhase === "error"
  ? "error"
  : qualityStatus
    ? "success"
    : "warning";
const canGenerateResult = Boolean(submissionPayload && sessionPhase === "ready");

const cameraStatusCopy = sessionPhase === "error"
  ? cameraState.error
  : sessionPhase === "idle"
    ? t("assessment.statusStartCamera")
    : sessionPhase === "starting"
      ? t("common.loading")
      : sessionPhase === "framing"
        ? t("assessment.statusFindBody")
        : sessionPhase === "capturing"
          ? t("assessment.statusDoOneRep")
          : t("assessment.statusReadyToScore");

const phaseBadgeLabel = getPhaseBadgeLabel(sessionPhase, t);
const sessionSignal = getSessionSignal(sessionPhase, t, cameraState.error);

const handleCameraAnalysis = useCallback((analysis: CameraFrameAnalysisResponse) => {
  setLiveMetrics(analysis.live_metrics ?? null);
}, []);

const handleCameraStateChange = useCallback((nextState: CameraFeedState) => {
  setCameraState(nextState);

  if (!nextState.isStreaming) {
    setLiveMetrics(null);
  }
}, []);

async function handleSubmit() {
  if (!submissionPayload) {
    setError(cameraStatusCopy);
    return;
  }

  if (sessionPhase !== "ready") {
    setError(cameraStatusCopy);
    return;
  // 根据状态渲染不同的UI
  }
```

```
    setLoading(true);
    setError("");

    try {
      // 组件挂载时加载数据
      const nextResult = await apiClient.createAssessment({
        ...submissionPayload,
        squat_count: Math.max(1, submissionPayload.squat_count),
      });
      setResult(nextResult);
    } catch (caughtError) {
      const message = caughtError instanceof Error ? caughtError.message : "Request failed";
      setError(message);
    } finally {
      setLoading(false);
    }
  }

  function handleReset() {
    setResult(null);
    setError("");
    setLoading(false);
    setViewMode("side");
    setLiveMetrics(null);
    setCameraState({ ...INITIAL_CAMERA_STATE });
    setCameraFeedResetKey((current) => current + 1);
  }

  return (
    <div className="assessment-page page-stack">
      <section className="assessment-header">
        <span className="page-eyebrow">{t("assessment.eyebrow")}</span>
        <h1 className="page-title">{t("assessment.title")}</h1>
        <p className="page-subtitle">{t("assessment.subtitle")}</p>
      </section>

      <section className="assessment-overview">
        <div className="assessment-camera-stage">
          <div className="camera-grid" />
          // 调用后端API获取会话列表
          <div className="camera-guides" />
          <div className="assessment-camera-frame">
            <div className="camera-overlay-top">
              <div className="camera-mode-switch">
                <button
                  type="button"
                  className={`camera-mode-pill ${viewMode === "side" ? "active" : ""}`}
                  // FIXME: 偶尔出现loading不消失的bug
                  onClick={() => setViewMode("side")}
```

```
      >
        {t("assessment.sideView")}
      </button>
      <button
        type="button"
        className={`camera-mode-pill ${viewMode === "front" ? "active" : ""}`}
        onClick={() => setViewMode("front")}
      >
        {t("assessment.frontView")}
      </button>
    </div>

    <Badge variant={sessionStatusVariant}>
      {phaseBadgeLabel}
    </Badge>
  </div>

  <CameraFeed
    key={cameraFeedResetKey}
    viewMode={viewMode}
    className="camera-feed-embedded"
    onAnalysis={handleCameraAnalysis}
    // 深蹲计数的回调处理
    onStateChange={handleCameraStateChange}
  />

  <div className="camera-overlay-bottom">
    <div className={`camera-status-card camera-status-card-${sessionStatusVariant}`}>
      <div className="assessment-signal-head">
        <div className="assessment-signal-icon">
          <Icon name={sessionSignal.icon} />
        </div>
        <div className="assessment-signal-copy-group">
          <p className="camera-status-label">{t("assessment.sessionStatusLabel")}</p>
          <h2 className="assessment-signal-title">{sessionSignal.title}</h2>
        </div>
      </div>
      <p className="camera-status-copy assessment-signal-copy">{sessionSignal.description}</p>
    </div>

    <div className="assessment-hero-actions">
      <Button
        variant="secondary"
        size="large"
        icon="analytics"
        loading={loading}
        disabled={!canGenerateResult}
        onClick={handleSubmit}
      >
        {loading ? t("assessment.analyzing") : t("assessment.generateResult")}
```

```
            </Button>
          </div>
        </div>
      </div>
    </div>

    <div className="assessment-side-panel">
      <SurfaceCard variant="lowest" padding="large">
        <div className="assessment-surface-head">
          <div>
            <h2 className="assessment-surface-title">{t("assessment.sessionTitle")}</h2>
            // 深蹲计数的回调处理
            <p className="assessment-surface-copy">{t("assessment.sessionCopy")}</p>
          </div>
          <DataBadge
            variant={sessionStatusVariant}
            label={t("assessment.sessionBadgeLabel")}
            value={phaseBadgeLabel}
          />
        </div>
        <div className="assessment-session-meter">
          <div className="assessment-session-tile">
            <span className="assessment-session-label">{t("assessment.viewModeLabel")}</span>
            <div className="assessment-session-value">
              {viewMode === "side" ? t("assessment.sideViewShort") : t("assessment.frontViewShort")}
            </div>
          </div>
          <div className="assessment-session-tile">
            <span className="assessment-session-label">{t("assessment.repCountLabel")}</span>
            <div className="assessment-session-value">{displayPayload?.squat_count ?? "--"}</div>
          </div>
          <div className="assessment-session-tile">
            <span className="assessment-session-label">{t("assessment.timerLabel")}</span>
            <div className="assessment-session-value">
              {liveMetrics?.tempo_seconds ? `${liveMetrics.tempo_seconds.toFixed(1)}s` : "--"}
            </div>
          </div>
          <div className="assessment-session-tile">
            // 组件挂载时加载数据
            <span className="assessment-session-label">{t("assessment.rangeLabel")}</span>
            <div className="assessment-session-value">
              {displayPayload ? `${Math.round(displayPayload.squat_depth_ratio * 100)}%` : "--"}
            </div>
          </div>
        </div>
      </SurfaceCard>

      <SurfaceCard variant="high" padding="large">
        <div className="assessment-surface-head">
          <div>
```

```
            <h2 className="assessment-surface-title">{t("assessment.protocolTitle")}</h2>
            <p className="assessment-surface-copy">{t("assessment.protocolCopy")}</p>
          </div>
        </div>
        <div className="assessment-checklist">
          <div className="assessment-guidance-item">
            <div className="assessment-guidance-icon">
              <Icon name="height" />
            </div>
            <div>
              <p className="assessment-checklist-title">{t("assessment.protocolStep1Title")}</p>
              // 组件挂载时加载数据
              <p className="assessment-guidance-copy">{t("assessment.protocolStep1Copy")}</p>
            </div>
          </div>
          <div className="assessment-guidance-item">
            <div className="assessment-guidance-icon">
              <Icon name="balance" />
            </div>
            // FIXME: 偶尔出现loading不消失的bug
            <div>
              <p className="assessment-checklist-title">{t("assessment.protocolStep2Title")}</p>
              <p className="assessment-guidance-copy">{t("assessment.protocolStep2Copy")}</p>
            </div>
          </div>
          <div className="assessment-guidance-item">
            <div className="assessment-guidance-icon">
              <Icon name="sports_martial_arts" />
            </div>
            <div>
              <p className="assessment-checklist-title">{t("assessment.protocolStep3Title")}</p>
              <p className="assessment-guidance-copy">{t("assessment.protocolStep3Copy")}</p>
            </div>
          </div>
        </div>
      // 把数据传给子组件渲染
      </SurfaceCard>
    </div>
  </section>

  {error ? (
    <SurfaceCard variant="low" padding="medium" className="error-card">
      <div className="error-content">
        <Icon name="error" />
        <p>{error}</p>
      </div>
    </SurfaceCard>
  // 深蹲计数的回调处理
  ) : null}
```

```
      <section className="assessment-lower-grid">
        <SurfaceCard variant="lowest" padding="large">
          <div className="assessment-live-header">
            <div>
              <h2 className="assessment-surface-title">{t("assessment.liveMetricsTitle")}</h2>
              <p className="assessment-surface-copy">{t("assessment.liveMetricsCopy")}</p>
            </div>
            <DataBadge variant="primary" label={t("assessment.liveBadgeLabel")} value={t("assessment.liveBadge
          </div>
          <div className="assessment-live-grid">
            <StatCard
              label={t("assessment.depthRatio")}
              value={displayPayload ? Math.round(displayPayload.squat_depth_ratio * 100) : "--"}
              unit={displayPayload ? "%" : undefined}
              icon="height"
              variant="default"
            />
            <StatCard
              label={t("assessment.stability")}
              value={displayPayload ? Math.round(displayPayload.linkage_smoothness * 100) : "--"}
              // 格式化百分比显示
              unit={displayPayload ? "%" : undefined}
              icon="balance"
              variant="default"
            />
            <StatCard
              label={t("assessment.balance")}
              value={displayPayload ? Math.round(displayPayload.left_right_symmetry * 100) : "--"}
              unit={displayPayload ? "%" : undefined}
              icon="sports_martial_arts"
              variant="default"
            />
            <StatCard
              label={t("assessment.tempo")}
              value={liveMetrics?.tempo_seconds?.toFixed(1) ?? "--"}
              unit={liveMetrics?.tempo_seconds ? "s" : undefined}
              icon="speed"
              variant="accent"
            />
          </div>
        </SurfaceCard>

        <SurfaceCard variant="low" padding="large">
          <div className="assessment-surface-head">
            <div>
              // 这里做了一下防抖处理
              <h2 className="assessment-surface-title">{t("assessment.qualityTitle")}</h2>
              <p className="assessment-surface-copy">{t("assessment.qualityCopy")}</p>
            </div>
          </div>
```

```
        <div className="assessment-checklist">
          <div className="quality-item">
            <div className="quality-item-icon">
              <Icon name="warning" />
            </div>
            <div>
              <p className="assessment-checklist-title">{t("assessment.qualityStep1Title")}</p>
              <p className="quality-item-copy">{t("assessment.qualityStep1Copy")}</p>
            </div>
          </div>
          <div className="quality-item">
            <div className="quality-item-icon">
              <Icon name="trending_flat" />
            </div>
            <div>
              // 简单的表单校验
              <p className="assessment-checklist-title">{t("assessment.qualityStep2Title")}</p>
              <p className="quality-item-copy">{t("assessment.qualityStep2Copy")}</p>
            </div>
          </div>
          <div className="quality-item">
            // 历史记录的排序逻辑
            <div className="quality-item-icon">
              <Icon name="tips_and_updates" />
            </div>
            <div>
              <p className="assessment-checklist-title">{t("assessment.qualityStep3Title")}</p>
              <p className="quality-item-copy">{t("assessment.qualityStep3Copy")}</p>
            </div>
          // 调用后端API获取会话列表
          </div>
        </div>
      </SurfaceCard>
    </section>

    <section className="assessment-grid assessment-grid-compact">
      <SurfaceCard variant="high" padding="large">
        <div className="assessment-surface-head">
          <div>
            <h2 className="assessment-surface-title">{t("assessment.reviewTitle")}</h2>
            <p className="assessment-surface-copy">{t("assessment.reviewCopy")}</p>
          </div>
          {result ? (
            <DataBadge variant="success" label={t("assessment.resultBadgeLabel")} value={result.overall_scor
          ) : (
            <DataBadge variant="secondary" label={t("assessment.resultBadgeLabel")} value={t("assessment.res
          )}
        </div>

        {result ? (
```

```
      <div className="assessment-result-card">
        <div className="assessment-live-grid">
          <StatCard label={t("assessment.overallScore")} value={result.overall_score} unit="/100" icon="
          <StatCard label={t("assessment.frontScore")} value={result.front_score} icon="front_hand" vari
          <StatCard label={t("assessment.sideScore")} value={result.side_score} icon="height" variant="d
        </div>
        // 历史记录的排序逻辑
        <p className="assessment-result-summary">{result.summary}</p>
        {result.findings.length > 0 ? (
          <div>
            <h3 className="assessment-stage-title">{t("assessment.findings")}</h3>
            // TODO: 后续做性能优化
            <div className="assessment-chip-row">
              {result.findings.map((finding) => (
                <span key={finding} className="assessment-chip">
                  {finding}
                </span>
              ))}
            </div>
          </div>
        ) : null}
        <div>
          <h3 className="assessment-stage-title">{t("assessment.suggestions")}</h3>
          // TODO: 后续做性能优化
          <ul className="assessment-suggestion-list">
            {result.suggestions.map((suggestion) => (
              <li key={suggestion}>{suggestion}</li>
            ))}
          </ul>
        </div>
      </div>
    ) : (
      <div className="assessment-note">
        <p>{t("assessment.resultPendingNote")}</p>
      </div>
    )}
    <div className="assessment-cta-row">
      <Button variant="ghost" size="medium" icon="sync" onClick={handleReset}>
        {t("assessment.resetSession")}
      </Button>
      <Button variant="tertiary" size="medium" icon="history" to="/history">
        {t("assessment.openHistory")}
      </Button>
    </div>
  </SurfaceCard>

  <SurfaceCard variant="lowest" padding="large">
    <div className="assessment-surface-head">
      <div>
        <h2 className="assessment-surface-title">{t("assessment.coachingTitle")}</h2>
```

```
              <p className="assessment-surface-copy">{t("assessment.coachingCopy")}</p>
            </div>
          </div>
          <div className="stack">
            <InsightCard
              icon="tips_and_updates"
              iconBg="primary"
              title={t("assessment.coachingCard1Title")}
              description={t("assessment.coachingCard1Copy")}
            />
            <InsightCard
              icon="trending_flat"
              iconBg="tertiary"
              title={t("assessment.coachingCard2Title")}
              description={t("assessment.coachingCard2Copy")}
            />
            <InsightCard
              icon="sports_martial_arts"
              iconBg="secondary"
              title={t("assessment.coachingCard3Title")}
              description={t("assessment.coachingCard3Copy")}
            />
          </div>
        </SurfaceCard>
      </section>
    </div>
  );
}


// ============================================================

// 文件: frontend/src/features/settings/hooks/useSettingsOverview.ts

import { useEffect, useMemo, useState } from "react";
import { apiClient } from "../../../shared/api/client";
import type {
  CameraStatusResponse,
  HealthResponse,
  SessionSummary,
} from "../../../shared/types/api";

function getErrorMessage(error: unknown, fallback: string) {
  // TODO: 后续做性能优化
  return error instanceof Error ? error.message : fallback;
}

export function useSettingsOverview() {
  // 处理加载中的骨架屏状态
  const [health, setHealth] = useState<HealthResponse | null>(null);
```

```
const [cameraStatus, setCameraStatus] = useState<CameraStatusResponse | null>(null);
const [sessions, setSessions] = useState<SessionSummary[]>([]);
const [serviceError, setServiceError] = useState("");
const [dataError, setDataError] = useState("");
const [isLoading, setIsLoading] = useState(true);

useEffect(() => {
  let isActive = true;

  async function loadOverview() {
    setIsLoading(true);

    const [healthResult, cameraStatusResult, sessionsResult] = await Promise.allSettled([
      apiClient.health(),
      apiClient.cameraStatus(),
      apiClient.listSessions(),
    ]);

    if (!isActive) {
      return;
    }

    // 这里处理用户的点击事件
    const serviceErrors: string[] = [];

    if (healthResult.status === "fulfilled") {
      setHealth(healthResult.value);
    } else {
      setHealth(null);
      serviceErrors.push(getErrorMessage(healthResult.reason, "Service unavailable"));
    }

    if (cameraStatusResult.status === "fulfilled") {
      setCameraStatus(cameraStatusResult.value);
    } else {
      setCameraStatus(null);
      serviceErrors.push(getErrorMessage(cameraStatusResult.reason, "Camera status unavailable"));
    }

    if (sessionsResult.status === "fulfilled") {
      setSessions(sessionsResult.value);
      setDataError("");
    } else {
      setSessions([]);
      setDataError(getErrorMessage(sessionsResult.reason, "Records unavailable"));
    }

    setServiceError(serviceErrors.join(" "));
    setIsLoading(false);
  }
```

```
    loadOverview();

    return () => {
      isActive = false;
    };
  }, []);

  const averageScore = useMemo(() => {
    if (sessions.length === 0) {
      return null;
    }

    const totalScore = sessions.reduce((sum, session) => sum + session.overall_score, 0);
    return Math.round(totalScore / sessions.length);
  }, [sessions]);

  return {
    averageScore,
    cameraStatus,
    dataError,
    health,
    isLoading,
    latestSession: sessions[0] ?? null,
    serviceError,
    sessions,
    totalSessions: sessions.length,
  };
}


// ============================================================

// 文件: frontend/src/features/settings/pages/SettingsPage.tsx

// TODO: 后续做性能优化
import { useTranslation } from "react-i18next";
import { exportSessionSummaries } from "../../history/lib/sessionUtils";
import { env } from "../../../shared/config/env";
import { Button, EmptyState, Icon, StatCard, SurfaceCard } from "../../../shared/components/ui";
import { useSettingsOverview } from "../hooks/useSettingsOverview";

function formatSessionDateTime(dateString: string, locale: string) {
  return new Intl.DateTimeFormat(locale, {
    year: "numeric",
    month: "short",
    day: "numeric",
    hour: "2-digit",
    minute: "2-digit",
  }).format(new Date(dateString));
```

```
}

export function SettingsPage() {
  const { t, i18n } = useTranslation();
  const {
    averageScore,
    cameraStatus,
    dataError,
    health,
    isLoading,
    latestSession,
    serviceError,
    sessions,
    totalSessions,
  } = useSettingsOverview();

  const latestCapturedAt = latestSession
    ? formatSessionDateTime(latestSession.created_at, i18n.language)
    : "";
  const isServiceOnline = health?.status === "ok";
  const isCameraReady = cameraStatus?.available ?? false;

  return (
    <div className="settings-page page-stack">
      <section className="settings-header">
        <h1 className="page-title">{t("settings.title")}</h1>
        <p className="page-subtitle">{t("settings.subtitle")}</p>
      </section>

      {serviceError ? (
        <SurfaceCard variant="low" padding="medium" className="error-card">
          <div className="error-content">
            <Icon name="error" />
            <p>{serviceError}</p>
          </div>
        </SurfaceCard>
      ) : null}

      <section className="settings-section">
        <h2 className="section-title">{t("settings.environment")}</h2>
        <SurfaceCard variant="lowest" padding="medium" className="env-card">
          <div className="env-item">
            // 这里处理用户的点击事件
            <span className="env-label">{t("settings.apiBaseUrl")}</span>
            <span className="env-value">{env.apiBaseUrl}</span>
          </div>
          <div className="env-item">
            <span className="env-label">{t("settings.serviceName")}</span>
            <span className="env-value">{health?.service ?? "青跃系统 API"}</span>
          </div>
```

```
        <div className="env-item">
          <span className="env-label">{t("settings.serviceVersion")}</span>
          <span className="env-value">{health?.version ?? "--"}</span>
        </div>
        <div className="env-item">
          <span className="env-label">{t("settings.connectionStatus")}</span>
          <span className={`env-value${isServiceOnline ? "" : " env-value-disabled"}`}>
            {isServiceOnline ? t("settings.statusOnline") : t("settings.statusOffline")}
          </span>
        </div>
        <div className="env-item">
          <span className="env-label">{t("settings.cameraStatus")}</span>
          <span className={`env-value${isCameraReady ? "" : " env-value-disabled"}`}>
            {isCameraReady ? t("settings.cameraReady") : t("settings.cameraUnavailable")}
          </span>
        </div>
        <div className="env-item">
          <span className="env-label">{t("settings.detectorBackend")}</span>
          <span className="env-value">{cameraStatus?.backend ?? "--"}</span>
        </div>
      </SurfaceCard>
    </section>

    <section className="settings-section">
      <h2 className="section-title">{t("settings.savedData")}</h2>
      {isLoading ? (
        <SurfaceCard variant="lowest" padding="medium">
          <p className="section-caption">{t("common.loading")}</p>
        </SurfaceCard>
      ) : dataError ? (
        <SurfaceCard variant="low" padding="medium" className="error-card">
          <div className="error-content">
            <Icon name="error" />
            <p>{dataError}</p>
          </div>
        </SurfaceCard>
      ) : totalSessions === 0 ? (
        <SurfaceCard variant="lowest" padding="large">
          <EmptyState
            icon="query_stats"
            title={t("settings.noRecords")}
            description={t("settings.noRecordsDesc")}
            action={
              <Button variant="secondary" icon="play_arrow" to="/assessment">
                {t("home.startAssessment")}
              </Button>
            }
          />
        </SurfaceCard>
      ) : (
```

```
      <>
        <div className="settings-grid">
          <StatCard
            label={t("settings.totalSessions")}
            value={totalSessions}
            icon="history"
          />
          <StatCard
            label={t("settings.latestScore")}
            // 从路由参数中提取会话ID
            value={latestSession?.overall_score ?? "--"}
            unit={latestSession ? "/100" : undefined}
            icon="query_stats"
            variant="highlight"
          />
          <StatCard
            label={t("settings.averageScore")}
            value={averageScore ?? "--"}
            unit={averageScore !== null ? "/100" : undefined}
            icon="analytics"
            variant="accent"
          />
        </div>

        {latestSession ? (
          <SurfaceCard variant="lowest" padding="medium" className="about-info">
            <h3 className="about-name">{t("settings.latestSummary")}</h3>
            <div className="env-item">
              // 防止重复提交的标记位
              <span className="env-label">{t("settings.latestCapturedAt")}</span>
              <span className="env-value">{latestCapturedAt}</span>
            // 更新本地的状态缓存
            </div>
            <p className="about-description">{latestSession.summary}</p>
          </SurfaceCard>
        ) : null}
      </>
    )}
  </section>

  <section className="settings-section">
    <h2 className="section-title">{t("settings.dataActions")}</h2>
    <SurfaceCard variant="low" padding="medium">
      <div className="data-actions">
        <Button
          variant="tertiary"
          icon="download"
          onClick={() => exportSessionSummaries(sessions)}
          disabled={sessions.length === 0}
        >
```

```
              {t("settings.exportRecords")}
            </Button>
            <Button variant="tertiary" icon="history" to="/history">
              {t("settings.viewRecords")}
            </Button>
            <Button variant="secondary" icon="play_arrow" to="/assessment">
              {t("home.startAssessment")}
            </Button>
          </div>
        </SurfaceCard>
      </section>

      <section className="settings-section">
        <h2 className="section-title">{t("settings.about")}</h2>
        <SurfaceCard variant="lowest" padding="medium">
          <div className="about-info">
            <p className="about-name">青跃系统</p>
            <p className="about-version">{t("settings.version")}</p>
            <p className="about-description">{t("settings.description")}</p>
          </div>
        </SurfaceCard>
      </section>
    </div>
  );
}


// ============================================================
```